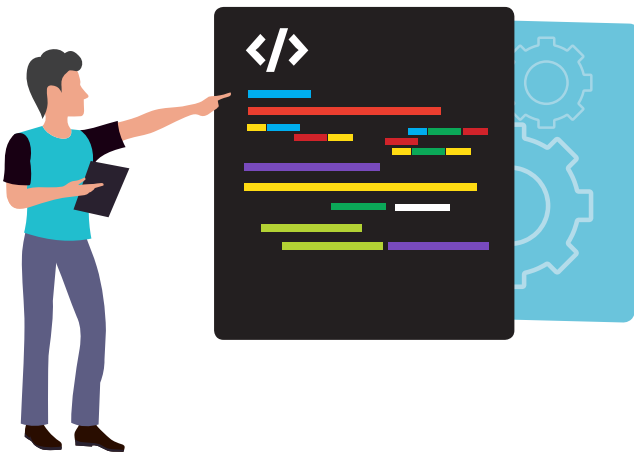


Code Generation: How the Lazy Become Prolific

Code generators bring the fun back into coding. This article lists their benefits and tells how they can be used in different ways, efficiently and effectively.



What does an enthusiastic programmer dream of? Getting to spend every day trying to solve real-world problems using interesting concepts and clever algorithms. And what do we really do as professional developers? Spend hours on a vast set of boring and artificial problems like npm dependency issues.

One of these boring and annoying problems is the creation and maintenance of boilerplate code. That's when we start exploring code generators. They first help us, then bring the fun back in, and transport us to a whole different frontier. What's more interesting? This is a rare case of the lazy becoming prolific!

Let's see what code generators can offer, where they are used, and how to make their use effective and efficient.

Code generators

What exactly do we mean by code generators? Compilers are the most basic kinds of code generators, generating Assembly or machine code from high-level language instructions. There are in-language code generation features too, like macros and generics. However, our focus will be on the kinds like preprocessors and transpilers, which generally convert between high-level languages, and those which generate programs or snippets from the declarative specifications that we give in YAML or JSON.

Code generation or better language?

Isn't code generation like a workaround? Why not pick a supreme language and use it directly instead of generating code in multiple stages?

First, the goal of code generation is not always to overcome language limitations. Just think about auto-generating HTML documentation for your API. You are not trying to overcome the limitation of Go or PHP here, right?

Second, general-purpose languages will always have tradeoffs or weak areas and you'll have to do something about it. Think of Go before generics. Think of powerful languages with unfriendly syntaxes. And even if there is a superior language, what if you are working on an existing codebase?

Also, code generators tend to be problem-specific. Not general-purpose, not even domain-specific, but problem-specific. That makes them so effective.

Finally, who said code generators cannot be languages? There is m4, a macro processor that is Turing-complete, and there is PHP, a templating system, which everybody considers to be more of a programming language.

The advantages of generative programming span from the ease and speed of development to maintainability, consistency, and memory safety and security. They can also be handy with shared code where librarification is hard or less efficient (microservices, maybe?). And then there is fun.

Too much talk. Let's see some examples.

Example: OpenAPI

Consider writing an API server. You not only have to write the handlers, but add code for routing, security checks, parameter extraction, validation, etc, which most of the time don't have anything to do with business logic. But things don't end there. You also have to provide documentation and client SDKs in various languages, keep them updated and in sync.

This is where OpenAPI comes in. With OpenAPI, you specify your API in a YAML or JSON file, and then use different generators to generate boilerplate code, client SDKs, documentation, etc. All you have to write manually is the business logic (well, most of the time).